Cockpit

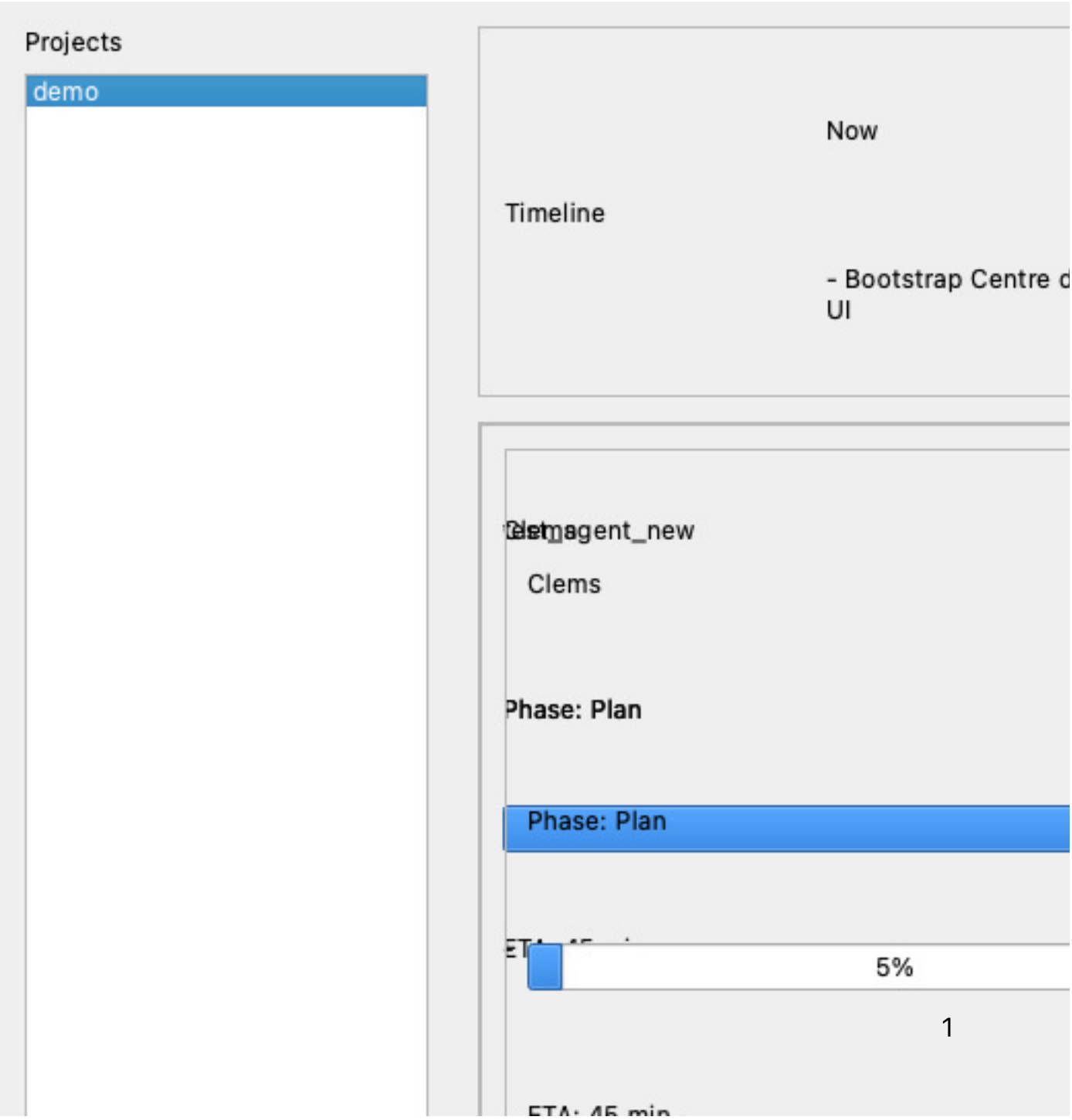
A visual, operator-friendly overview (with the Memory V2 direction).

project: cockpit phase: Implement generated: 2026-02-12 18:34 UTC

One-liner
Cockpit is a control tower to run a software project with multiple AI agents, with one orchestrator (Clems) keeping plan, guardrails, and accountability.

Current objective
V3.5.2 gate execution on cockpit (24h stability gate before rollout).

Canonical data dir
/Users/oliviercloutier/Library/Application Support/Cockpit/projects



1) Roles (team model)

Humans

- **Operator**: you. Sets goals, validates big decisions.

Leads (stable personas)

- **Clems**: orchestrator (plan, delegation, gatekeeping, decisions).
- **Victor**: backend lead (run-loop, storage, packaging, tests).
- **Leo**: UI lead (clarity, UX, QA).

Specialists (per project)

- Named strictly: **agent-1**, **agent-2**, ...
- Small missions, strict scope, verifiable Done.
- Work is delegated by Victor/Leo, supervised by Clems.

Rule of thumb: "No 12 random AIs." We run an actual org chart.

2) Project storage (local-first)

Cockpit stores everything locally per project. The app reads/writes only inside the projects root.

```
~/Library/Application Support/Cockpit/projects/<project_id>/
  STATE.md
  ROADMAP.md
  DECISIONS.md
  settings.json
  chat/
    global.ndjson
    threads/*.ndjson
  agents/<agent_id>/
    state.json
    memory.md
    journal.ndjson
  runs/
    requests.ndjson
    inbox/*.ndjson
    auto_mode_state.json
```

3) Automation pipeline (the core loop)

A mention in chat becomes a run request. Auto-mode routes it to an agent inbox. Auto-send (optional) executes it by pasting into the right external thread.

Operator message
(@victor/@leo/@agent-N)

→

runs/requests.ndjson
(queue)

→

AUTO-MODE
dispatch_once()

→

runs/inbox/<agent>.ndjson
AUTO-SEND OFF
(dispatch only)

↔

AUTO-SEND ON
(open app + paste + enter)

→

Agent replies via MCP

→

Lifecycle close
(reply_received)

Why "apps pop but nothing runs" happens

- If **AUTO-SEND is OFF**: we queue/dispatch only. No keystrokes are sent.
- If macOS blocks keystrokes: Accessibility permissions are missing.
- If thread naming drifts: window matching can skip sends (safety guardrail).

4) Where we are (roadmap)

We are running the V3.5.2 gate on project **cockpit**. The gate is 24h of real stability.

Plan
Implement
Test
Review
Ship

Gate pass criteria (24h)

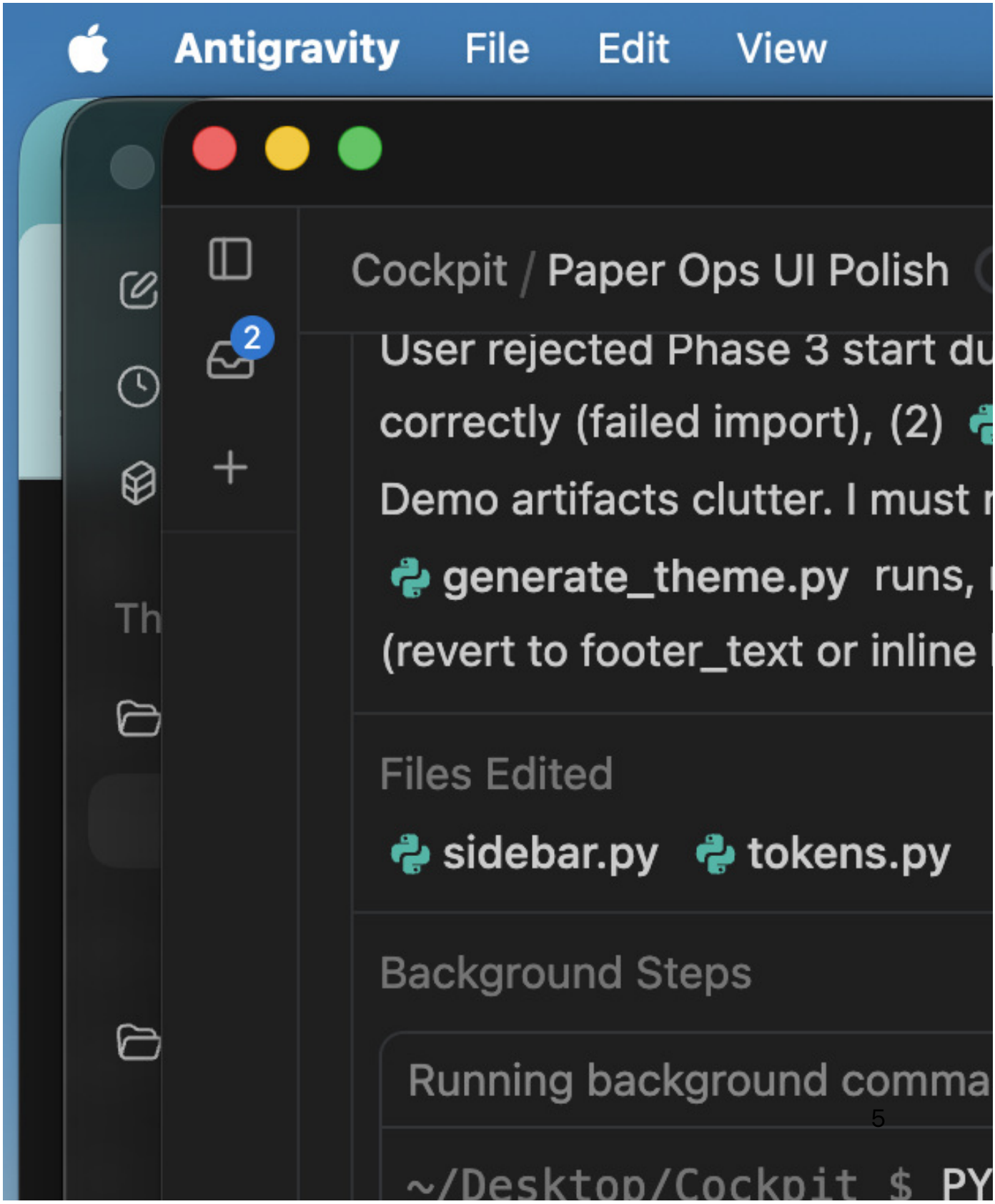
- close_rate_24h >= 0.80

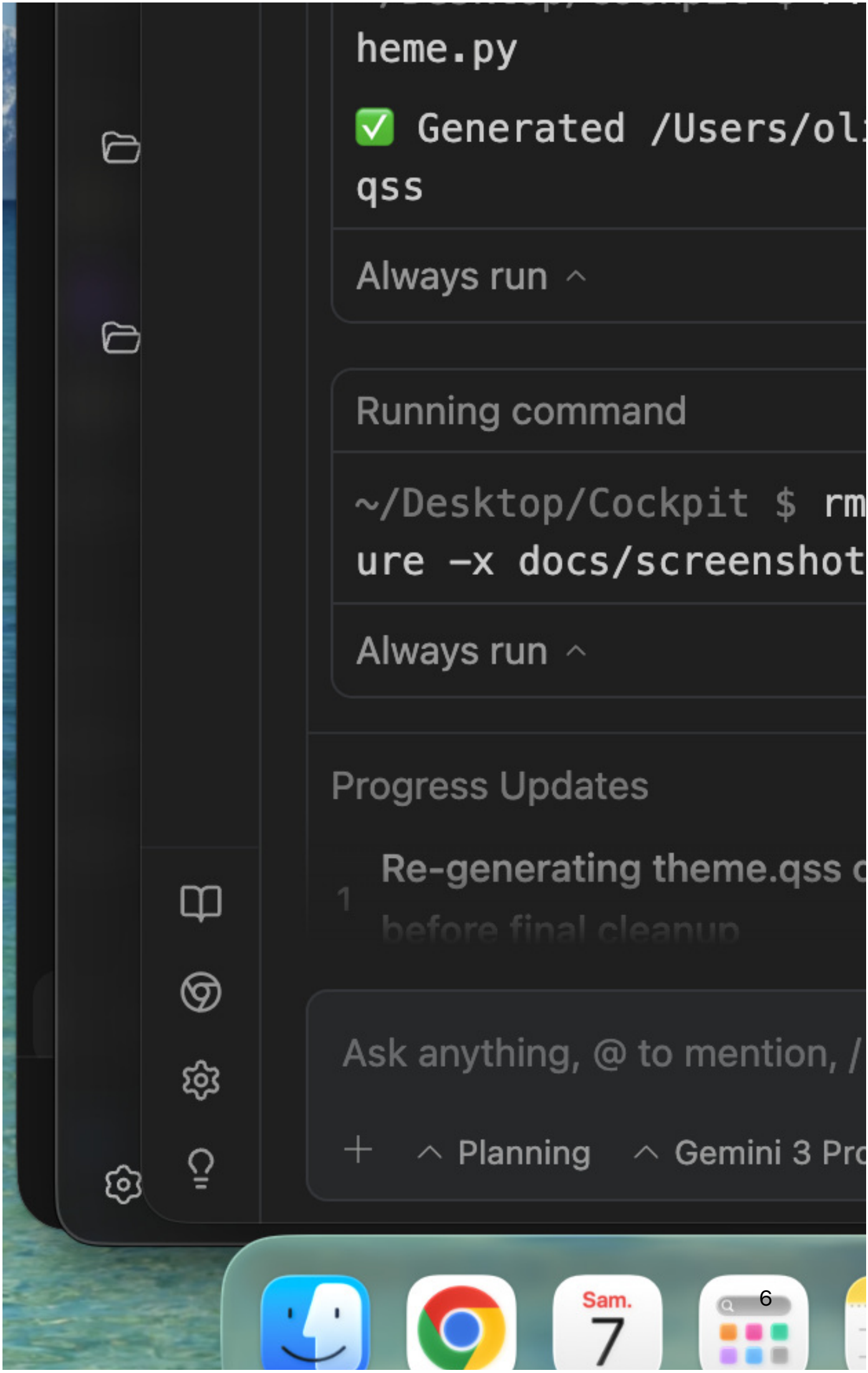
- reminder_noise_pct < 0.05
- wrong_project_leak = 0
- send_fail_rate < 0.10 (excluding permission-denied setup)

Rollout order after gate: cockpit → evozina → motherload

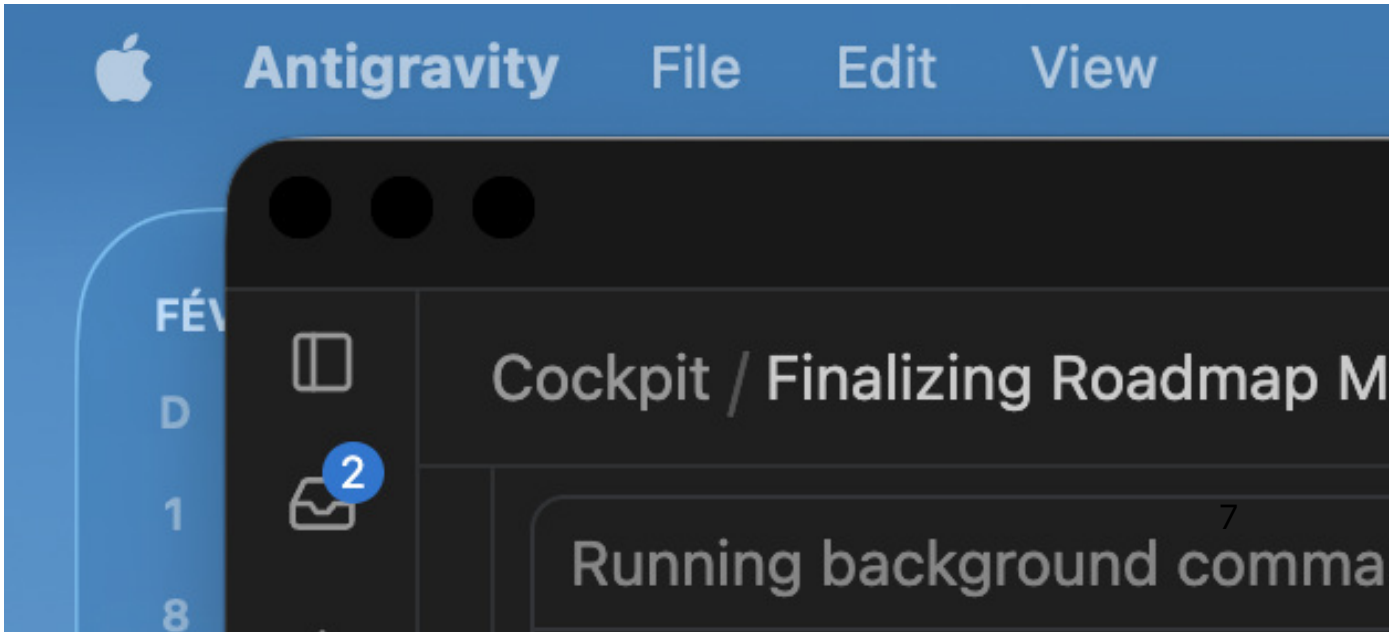
Visual reference (UI snapshots)

Paper Ops UI baseline (UI polish snapshot)




heme.py
✅ Generated /Users/ol
qss
Always run
Running command
~/Desktop/Cockpit $ rm
ure -x docs/screenshot
Always run
Progress Updates
1 Re-generating theme.qss c
before final cleanup
Ask anything, @ to mention, /
Planning
Gemini 3 Pro
Sam.
7

Automation supervision snapshot



15

22

Da

CO

```
~/Desktop/Cockpit $ sc
r_ops_roadmap_final_fr
```

Always run ^

Progress Updates

Recreating `generate_theme`

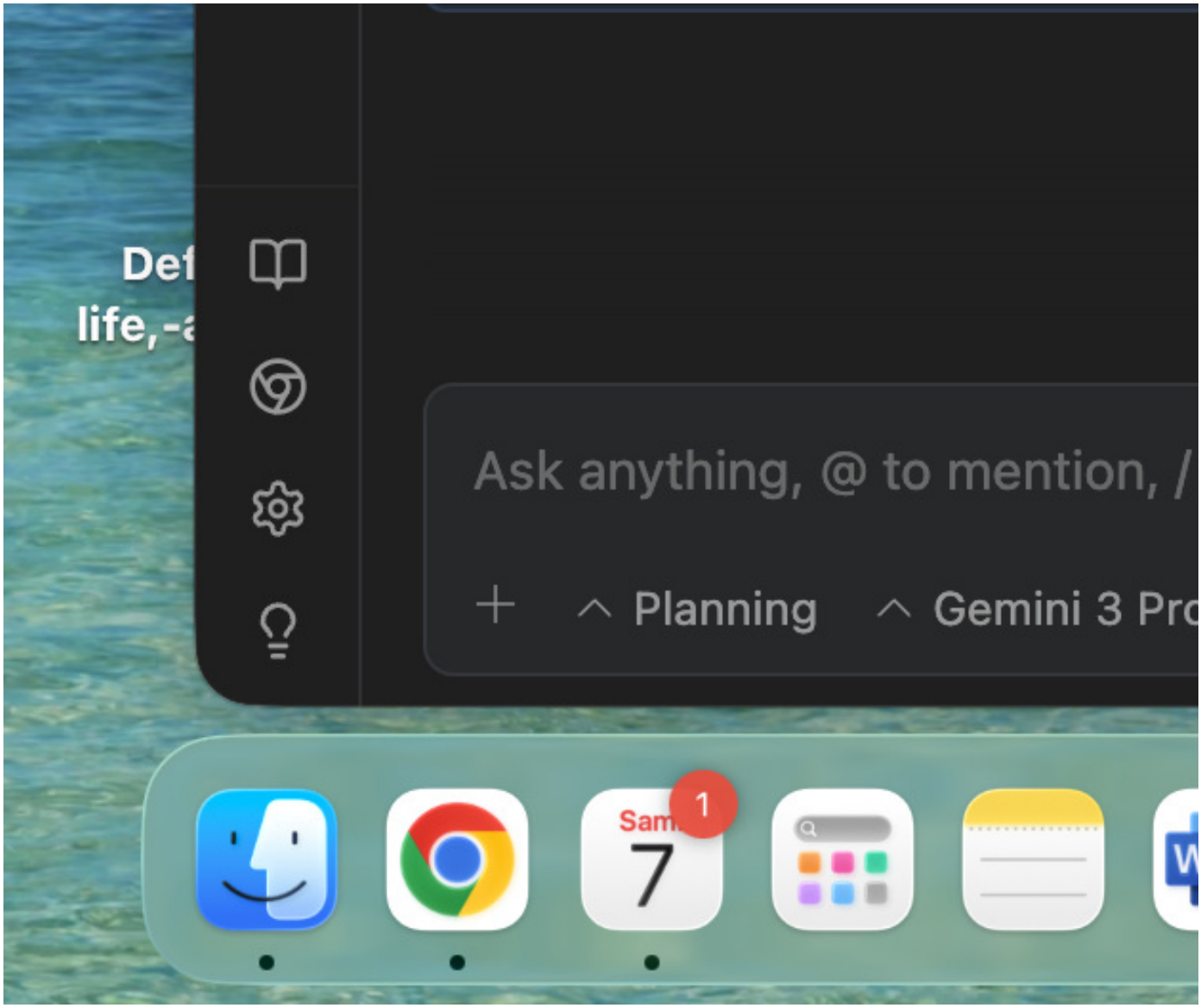
1 `file or directory` error, t

proof screenshot.

🇫🇷 **Mission Accomplie : Road**

1. **Microcopy (FR)** : `MISSION`
2. **Implementation** :
   - `roadmap.py` : Layout
   - `styles.py` : Restaur
   - `theme.qss` : Régéné
3. **Preuve** : `docs/screenshot`

Le code est en place. QA Visu


Def
life,-a
Ask anything, @ to mention, /
Planning
Gemini 3 Pro
Sam
7
1

5) Memory direction (review of agent-9 + Leo notes)

We want per-project memory that is useful daily, but safe (no magical rewriting). The current proposal lives in:
`control/projects/cockpit/PLAN_PAPER_SKILLS_OPS_MEMORY_V2.md`

Memory V1 / V2 / V3 (simple)

- **V1**: memory.md per agent, curated manually.
- **V2**: compaction tool proposes `memory.proposed.md` (human reviews), plus `memory.index.json`.
- **V3**: retrieval (RAG) later, strict cross-project isolation.

chat/global.ndjson
journal.ndjson
STATE/ROADMAP/DECISIONS

→

compact_memory
(propose, capped)

→

memory.proposed.md
(draft)

→

Human review
(accept/edit)

→

memory.md (canon)
memory.index.json

What is solid in the plan

- Non-destructive: never auto-overwrite `memory.md`.
- Determinism + caps: keeps output stable and small.
- Scope lock: cockpit only (no cross-project retrieval).
- Storage is explicit and auditable (files you can open).

What we corrected / want clarified

- Paths should be written in terms of `<projects_root>/<project_id>` (runtime is App Support by default).
- Compaction caps should be explicitly enforced (lines, per-section items, truncation).
- Index schema should be minimal but stable (facts/decisions/open_loops/links).

Suggested next step for memory (after gate)

- Implement `memory.index.json` generator (per agent).
- Wire compaction output to a one-click UI action (still non-destructive).
- Add 1-2 tests: determinism and caps.

6) Operator workflow (60 seconds)

1. Open Cockpit app, select project **cockpit**.
2. Ensure AUTO-MODE = ON.
3. If you want real execution: AUTO-SEND = ON.
4. Click Preflight, then Run once.
5. In chat, write a mission: `@victor ...` or `@leo ...` or `@agent-1 ...`
6. Watch: Pending goes down after replies; cards update; requests close with reason `reply_received`.

End of doc.